

Lecture 0: Big Data Analytics: Introduction & Platforms

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A

amitkdhar iitbhlai

January 7, 2026

- **Lectures:** Do Attend! NOT Optional.
- **Labs:** DSP352
- **Quizzes:** Announced and Unannounced. 2 OR 3
- **Mid-, End-Sem:** Highest weightage.

* Subject to Change

* May include a possibility of a project

Mid-Sem:	30%
End-Sem:	45%
Quizzes:	20%
Attendance:	5%

Traditional Database Systems

Relational Database Management Systems (RDBMS)

- Dominant data management paradigm for 40+ years.
- Based on the Relational Model (Codd, 1970).
- Data organized in tables (relations) with rows and columns.
- Strong schema enforcement.

Examples: MySQL, PostgreSQL, Oracle, SQL Server

Pros of Traditional RDBMS

- **ACID Properties:** Atomicity, Consistency, Isolation, Durability ensures data integrity.
- **Standardization:** SQL (Structured Query Language) is universal.
- **Complex Queries:** Powerful joins, aggregations, and sub-queries.
- **Security:** Mature access control and security features.
- **Transactional Support:** Essential for banking, e-commerce, etc.

Cons of Traditional RDBMS

- **Rigidity:** Schema on Write. Changing schema is difficult and downtime-prone.
- **Scalability:** Primarily designed for *Vertical Scaling* (Scale Up).
- **Cost:** High-end hardware (vertical scaling) becomes exponentially expensive.
- **Performance:** Joins become slow as data volume grows into Terabytes/Petabytes.
- **Variety:** Poor handling of unstructured data (JSON, text, media).

The Scaling Problem

Vertical Scaling (Scale Up)

- Add more RAM, CPU to a single server.
- **Limit:** Hardware ceiling.
- **Cost:** Expensive.

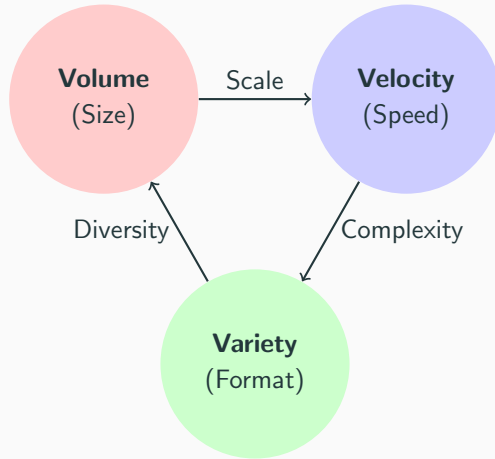
Horizontal Scaling (Scale Out)

- Add more commodity servers.
- **Limit:** Network complexity, Consistency issues.
- **Cost:** Linear, Cheaper.

RDBMS struggles to scale out efficiently while maintaining ACID.

Internet Scale Requirements

The 3 Vs of Big Data



Volume: Internet Scale Stats (2025/2026)

Data Generated Per Minute:

- **Global Traffic:** ~10 Petabytes.
- **Emails:** ~241 Million sent.
- **Google Searches:** ~11.4 Million.
- **YouTube:** ~3.5 Million video views.
- **Social Media:** Millions of posts, snaps, and shares.

Forecast: Global data generation to reach **221 Zettabytes** in 2026.

Real-time Ingestion Requirements:

- **IoT Sensors:** Jet engines generate TBs of data per flight.
- **Financial Markets:** Millions of transactions per second (High Frequency Trading).
- **Clickstreams:** Tracking user behavior on websites in real-time.
- **Cybersecurity:** Analyzing network packets for intrusion detection instantly.

Variety: Data Formats

Structured

- Relational Tables
- CSV, Excel
- Transaction logs

Unstructured / Semi-structured

- Text (Documents, Emails)
- JSON / XML
- Images, Video, Audio
- Sensor logs

80% of enterprise data is unstructured.

Calculation: The "Huge" Requirement

Scenario: Smart City Sensor Data

- 1 Million Sensors (Traffic, Weather, Pollution).
- Each sends 1 KB packet every 10 seconds.

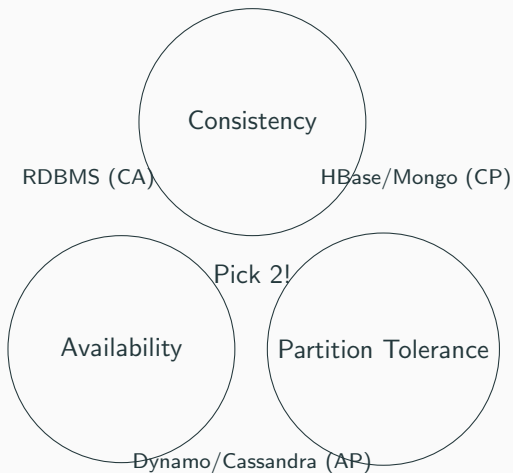
Calculation:

- Per Minute: $10^6 \times 6 \text{ packets} \times 1 \text{ KB} = 6 \text{ GB}$
- Per Day: $6 \text{ GB} \times 60 \times 24 \approx 8.6 \text{ TB}$
- Per Year: $8.6 \text{ TB} \times 365 \approx \mathbf{3.1 \text{ Petabytes}}$

Storing and querying 3.1 PB in a single MySQL instance is impossible.

Why Traditional Systems Fail

The CAP Theorem



In distributed systems (Partition Tolerance is mandatory), you must trade off Consistency or Availability.

- **RDBMS Choice:** CA (Consistency + Availability).
- **Assumption:** Network never partitions (Single Server).
- **Failure Mode:** When distributed, if the network splits, RDBMS cannot guarantee both ACID and Uptime.
- **Internet Scale:** Partition Tolerance is a fact of life (Networks fail). Thus, we must drop C or A.

Enterprise SAN vs. Commodity Hardware

- **Traditional DB:** Requires high-end Enterprise Storage (SAN/NAS).
 - Cost: \$10,000 - \$50,000 per TB.
- **Big Data approach:** Uses "Commodity Hardware" (standard servers with local disks).
 - Cost: \$50 - \$100 per TB.

For Petabyte scale, traditional storage is economically unviable.

- **Scenario:** Adding a new field "Device_Type" to a table with 10 Billion rows.
- **RDBMS:** 'ALTER TABLE' locks the table. downtime could be hours or days.
- **Big Data (NoSQL/Hadoop):** Schema on Read.
 - Just dump the new JSON/Log format.
 - Parse the new field only when querying.
 - No downtime.

The Way Out: Big Data Platforms

Paradigm Shift

Traditional

- Bring Data to Code.
- Centralized Storage.
- Complex transformations in App.

Big Data

- **Bring Code to Data.**
- Distributed Storage (HDFS).
- Processing happens where data lives.

1. **Distributed File System (DFS):** Split huge files into "blocks" and spread them across thousands of machines. (e.g., GFS, HDFS)
2. **Replication:** Keep 3 copies of every block. If a node fails, data is safe.
3. **Parallel Processing:** Process data on all nodes simultaneously.

The Ecosystem Overview

Many tools for different needs:

- **Batch Processing:** Hadoop MapReduce
- **Speed/In-Memory:** Apache Spark
- **SQL Interfaces:** Hive, Presto
- **Real-Time:** Storm, Flink
- **NoSQL Storage:** HBase, Cassandra, MongoDB

Course Platforms



- **Components:** HDFS (Storage), YARN (Resource Management), MapReduce (Processing).
- **Role:** The foundation of the Big Data ecosystem.
- **Course Coverage:** Architecture, MapReduce programming, YARN scheduling.
- **Use Case:** Large scale batch processing, Data Lake storage.



- **Key Feature:** In-Memory Processing (100x faster than MapReduce).
- **Unified Engine:** Supports SQL, Streaming, ML (Machine Learning), and Graph processing.
- **Course Coverage:** Core RDDs, DataFrames, Spark SQL.
- **Use Case:** Iterative algorithms, Interactive queries, ETL pipelines.



- **Function:** Data Warehouse software on top of Hadoop.
- **Interface:** HiveQL (SQL-like dialect).
- **Mechanism:** Translates SQL queries into MapReduce/Spark jobs.
- **Course Coverage:** Tables, Partitions, Bucketing, Querying.
- **Use Case:** Analysts who know SQL but not Java/Scala.

Apache Storm

- **Type:** Real-time computation system.
- **Model:** Processes unbounded streams of data.
- **Key Concept:** Spouts (Sources) and Bolts (Processing logic).
- **Course Coverage:** Topology design for real-time analytics.
- **Use Case:** Real-time alerts, Fraud detection.

Apache Giraph

- **Type:** Iterative Graph Processing system.
- **Origin:** Built for high scalability (used by Facebook).
- **Model:** Vertex-centric programming (Bulk Synchronous Parallel).
- **Course Coverage:** Graph algorithms (PageRank, Shortest Path).
- **Use Case:** Social Network Analysis.

NoSQL Databases (Cassandra, MongoDB, HBase)



- **Cassandra:** Wide-column store, high availability, masterless. (Example: Netflix user history).
- **MongoDB:** Document store (JSON), flexible schema. (Example: CMS, Product Catalogs).
- **HBase:** Wide-column store on HDFS, strong consistency. (Example: Facebook Messaging).
- **Course Coverage:** Data models and basic querying.

Apache Pig

- **Type:** High-level scripting language (Pig Latin).
- **Goal:** Simplify complex MapReduce programming.
- **Course Coverage:** Lab ecosystem tool for data extraction and transformation.
- **Use Case:** ETL scripts, Ad-hoc data investigation.

Real Life Examples



- **Scale:** 300M+ subscribers, 15% of global internet traffic.
- **Problem:** Providing personalized recommendations in real-time.
- **Solution:**
 - **Kafka:** Ingests billions of events/day.
 - **Spark:** Computes recommendations (ML models).
 - **Cassandra:** Stores user viewing history across regions.

Uber: Real-Time Marketplace

- **Scale:** 12 Trillion messages/day.
- **Problem:** Matching riders to drivers, dynamic pricing (Surge), ETA calculation.
- **Solution:**
 - **HDFS/Hive:** Data Lake for historical analysis.
 - **Flink/Storm:** Real-time stream processing for pricing and fraud.
 - **Pinot:** Real-time analytics on fresh data.

Facebook (Meta): Social Graph

- **Scale:** 300+ Petabytes Data Warehouse.
- **Problem:** Serving the social graph and analytics for billions of users.
- **Solution:**
 - **Hadoop/Hive:** Massive batch processing of user logs.
 - **Presto:** Interactive SQL queries for data scientists.
 - **Giraph:** Graph processing for "People You May Know".

Questions?